

CADENCE

Especificación del lenguaje

Automatas, Teoria de Lenguajes y Compiladores

Proyecto Especial - Stage I: Diseno

Repositorio	https://github.com/fvillanueva1/tpe-tla
Equipo	Felipe Villanueva, Tadeo Gorganchian y Eduardo Tormakh
Salida principal	Archivo MIDI reproducible
Fecha	Agosto de 2026

Indice

1. 1. Equipo
2. 2. Repositorio
3. 3. Dominio
4. 4. Construcciones, prestaciones y alcance
5. 5. Casos de prueba
6. 6. Ejemplos de sintaxis
7. 7. Bibliografia

1. Equipo

Nombre	Apellido	Legajo	E-mail
Felipe	Villanueva	Pendiente	fvillanueva@itba.edu.ar
Tadeo	Gorganchian	65507	tgorganchian@itba.edu.ar
Eduardo	Tormakh	Pendiente	etormakh@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en la rama development del siguiente repositorio:

<https://github.com/fvillanueva1/tpe-tla>

3. Dominio

3.1. Problema y necesidad

La armonía tonal occidental organiza notas simultáneas en acordes y encadena esos acordes para producir tensión, reposo y dirección musical. Sin embargo, las herramientas habituales de composición exigen trabajar nota por nota sobre una grilla o un pentagrama. Operaciones musicales de alto nivel, como construir un acorde por su grado, modular una progresión o distribuir un acorde entre voces, quedan reducidas a tareas manuales, repetitivas y propensas a errores.

Cadence es un DSL para describir y transformar armonía tonal desde las abstracciones que utiliza un músico: tonalidades, grados, acordes, progresiones y voces. El compilador deduce las notas concretas, verifica restricciones musicales que pueden decidirse de forma estática y genera un artefacto musical reproducible.

3.2. Modelo computacional

El programa de entrada es un archivo textual escrito en Cadence. Su modelo computacional representa tonalidades, escalas, notas, intervalos, acordes, progresiones y voicings. Las dependencias del dominio son explícitas: una tonalidad determina una escala; un grado se interpreta dentro de una tonalidad; un acorde se construye sobre ese grado; una progresión ordena acordes; y un voicing asigna las notas de cada acorde a un conjunto de voces.

El lenguaje empleará tipado estático para detectar, antes de generar el artefacto final, combinaciones de valores y reglas musicales inválidas. El artefacto de salida principal será un archivo MIDI. La emisión de

partitura mediante LilyPond queda definida como extensión futura; no forma parte del compromiso mínimo de implementación.

3.3. Delimitación del dominio

- El lenguaje modela armonía tonal y conducción de voces; no pretende ser un secuenciador general ni un editor de audio.
- La compilación no controla sintetizadores ni hardware de audio. Produce archivos que pueden abrirse con herramientas de terceros.
- El análisis semántico validará reglas de dominio, por ejemplo notas, intervalos y grados válidos, pertenencia a tonalidad estricta, tesituras y movimientos prohibidos entre voces.

3.4. Diferenciación

Los lenguajes musicales revisados suelen construir composiciones de abajo hacia arriba: nota, evento o pista. Cadence invierte esa dirección: tonalidad, grado, acorde, progresión y voces. La nota puede ser declarada cuando haga falta, pero no es la abstracción central de una progresión armónica.

- Acordes diatónicos contruidos por grado dentro de una tonalidad.
- Progresiones como tipo de dato de primera clase, con operadores propios de concatenación, repetición y transposición.
- Modulación que preserva la función armónica de los grados.
- Conducción SATB con validación de quintas paralelas, octavas paralelas y cruce de voces.

4. Construcciones, prestaciones y alcance

4.1. Construcciones del lenguaje

8. Tipos del dominio: note, interval, key, scale, chord, progression y voicing; tipos generales integer, boolean y string; y vectores de dichos tipos.
9. Constantes de notas de C0 a B8, alteraciones, grados I a VII, intervalos musicales y duraciones.
10. Declaración de tonalidades con tónica y modo. Se incluyen mayor y menor como núcleo; los modos eclesiásticos quedan como extensión sintética planificada.
11. Obtención de la escala de una tonalidad y construcción de triadas o acordes de séptima por grado, por ejemplo triad on V of K.
12. Construcción de acordes por enumeración explícita de notas y de progresiones por grados, por ejemplo [I, V, vi, IV] in K.
13. Operadores + y - para transponer notas, acordes y progresiones; + para concatenar progresiones; y * para repetir una progresión.
14. Modulación hacia una tonalidad o relación tonal, preservando la función de cada acorde dentro de la progresión.

15. Voicing de progresiones como SATB y verificación de paralelismos de quinta, paralelismos de octava, cruce de voces y tesituras.
16. Operadores aritmeticos, relacionales y logicos; operador de pertenencia in; y acceso a propiedades como chord.root o progression.length.
17. Estructuras if-then-else, for y while; procedimientos reutilizables; comentarios de linea y bloque; y exportación de una progresión o voicing a MIDI con tempo en BPM.

4.2. Priorización de implementación

Nivel	Alcance
Nucleo comprometido	Tonalidades mayor/minor, acordes diatónicos por grado, progresiones, operadores + y *, validaciones semánticas básicas y generación de MIDI.
Diferencial prioritario	Voicing SATB, selección de inversiones de menor movimiento y detección de quintas paralelas, octavas paralelas y cruce de voces.
Extensiones planificadas	Partitura mediante LilyPond, arpeggios, modos adicionales, procedimientos, estructuras de control completas y construcciones auxiliares.

La especificación conserva las construcciones necesarias para expresar la visión completa del lenguaje.

La priorización reduce el riesgo de las etapas de implementación sin perder el diferencial del dominio.

5. Casos de prueba

Los siguientes casos son unitarios: cada uno comprueba una construcción o regla puntual. Los programas de aceptación deben ser analizados y aceptados; los de rechazo deben informar el error correspondiente.

5.1. Casos de aceptación

ID	Programa	Resultado esperado
A01	key K = C major;	Declara una tonalidad mayor válida.
A02	key K = C major; chord c = triad on V of K;	Construye la tríada dominante a partir de un grado.
A03	key K = C major; progression p = [I, V, vi, IV] in K;	Crea una progresion diatonica por grados.
A04	key K = C major; progression estrofa = [I, IV] in K; progression tema = estrofa * 2;	Repite una progresion.
A05	key K = C major; progression p = [I, V] in K; progression puente = modulate p to dominant of K;	Modula una progresion preservando su funcion armonica.
A06	key K = C major; boolean pertenece = E4 in scale of K;	Comprueba la pertenencia de una nota a una escala.
A07	chord c = {C4, E4, G4};	Declara un acorde por enumeración explícita.
A08	note e = C4 + M3;	Transporta una nota por un intervalo válido.
A09	key K = C major; progression p = [I, IV, V, I] in K; voicing v = voice p as SATB;	Distribuye una progresión válida entre cuatro voces y la verifica.

ID	Programa	Resultado esperado
A10	check v for parallel_fifths, parallel_octaves, voice_crossing; key K = C major; progression p = [I, V] in K; export p as midi to "tema.mid" at 120 bpm;	Emite una progresion valida como archivo MIDI.

5.2. Casos de rechazo

ID	Programa	Resultado esperado
R01	key K = C major	Rechazar por falta de punto y coma.
R02	note n = H4;	Rechazar porque H no es una nota válida.
R03	key K = C major; progression p = [VIII] in K;	Rechazar porque una escala diatónica posee siete grados.
R04	interval i = P3;	Rechazar porque una tercera no puede ser justa.
R05	Programa con un voicing de prueba que contiene quintas paralelas entre dos voces consecutivas.	Rechazar por quintas paralelas entre voces.

6. Ejemplos de sintaxis

Los ejemplos siguientes muestran las dos ideas centrales del lenguaje: definir progresiones por función armónica y trabajar con varias voces sin especificar manualmente cada nota intermedia.

6.1. Progresión por grados y exportacion MIDI

El siguiente programa construye la progresión I-V-vi-IV en Do mayor. El programador expresa grados; el compilador deduce los acordes concretos, repite la progresión y emite el archivo MIDI.

```
key DoMayor = C major;

progression estrofa = [I, V, vi, IV] in DoMayor;
progression tema = estrofa * 2;

export tema as midi to "tema.mid" at 120 bpm;
```

Codigo 6.1: Progresion I-V-vi-IV en Do mayor.

6.2. Conduccion de voces y validacion

Este programa construye una progresión cadencial en Sol mayor, la distribuye entre soprano, contralto, tenor y bajo, y solicita las validaciones de contrapunto definidas para el proyecto.

```
key Sol = G major strict;
```

```
progression coral = [I, IV, V, I] in Sol;  
voicing v = voice coral as SATB;  
  
check v for parallel_fifths, parallel_octaves, voice_crossing;  
export v as midi to "coral.mid" at 84 bpm;
```

Código 6.2: Conducción SATB con validación de contrapunto.

7. Bibliografía

18. ITBA. (2026). Proyecto Especial: Diseño e Implementación de un Lenguaje.
19. ITBA. (2026). Especificación: Diseño e Implementación de un Lenguaje.
20. ITBA. (2026). Modelo Computacional: Diseño e Implementación de un Lenguaje.
21. Piston, W. y DeVoto, M. (1987). Harmony (5.a ed.). W. W. Norton & Company.
22. The MIDI Association. General MIDI 1 Sound Set.
<https://www.midi.org/specifications-old/item/gm-level-1-sound-set>